SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA1001	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	M. Srilakshmi	**Reg. No.:**	192525404
Date:	26/10/2026	**Duration:**	60 minutes

Code of Conduct

I M. Srilakshmi (192525404) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M. Srilakshmi

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and	Diagram is incomplete, unclear, or

	major components, interfaces, and interactions accurately represented.		lacks sufficient detail.	technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

Predictive Analytics System for Early Detection of Land Acquisition Delays

1. Analyze System Requirements

1.1 Study the given problem statement and identify the system objectives.

The proposed system aims to predict possible delays in land acquisition projects before they become serious. It analyzes historical and current land acquisition data and provides early warnings to responsible officials.

The main objectives are:

- To collect and manage land acquisition information.
- To monitor the progress of land acquisition activities.
- To identify factors that may cause delays.
- To predict the probability of land acquisition delays.
- To classify acquisition cases as low, medium, or high risk.
- To generate early alerts for high-risk cases.
- To help officials take preventive actions.
- To provide dashboards and reports for decision-making.

1.2 Analyze the functional and non-functional requirements that influence the software architecture.

The system shall provide:

1. User registration and login.
2. Role-based user authentication.
3. Land record management.
4. Land acquisition progress tracking.
5. Compensation status management.
6. Legal issue and dispute tracking.
7. Approval status management.
8. Historical data storage.
9. Data validation and preprocessing.
10. Predictive analysis using machine learning.
11. Delay probability calculation.
12. Risk classification.
13. Early warning notifications.
14. Dashboard and report generation.
15. Search and filtering of acquisition records.
16. Audit logging.

Non-Functional Requirements

The system should provide:

- Scalability – Support increasing users, projects, and land records.
- Maintainability – Allow easy modification and updating.
- Reliability – Provide consistent and accurate results.
- Availability – Remain accessible when required.
- Performance – Provide quick dashboard and prediction results.
- Security – Protect landowner and project information.
- Usability – Provide a simple interface for officials.

2. Select a Suitable Software Architecture

2.1 Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).

The proposed system uses a Layered Architecture with Client-Server Architecture and a separate Predictive Analytics/ML component.

Main Layers

1. Presentation Layer
2. Application/Business Logic Layer
3. Predictive Analytics Layer
4. Data Access Layer
5. Database Layer
6. External Services Layer

2.2 Justify why the selected architecture is the most suitable for the given problem.

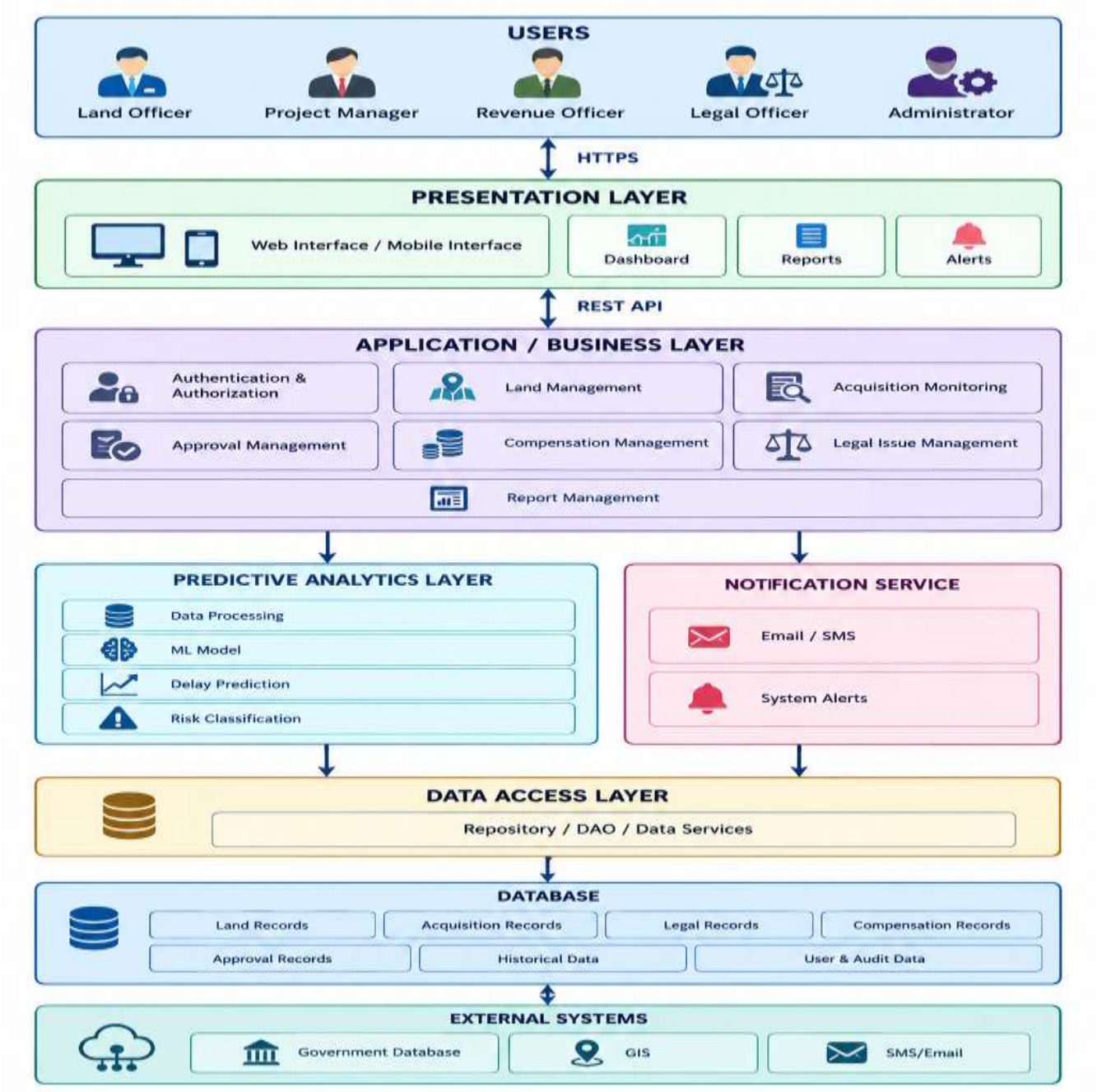
Layered architecture is suitable because the system has different responsibilities such as user interface, business processing, predictive analysis, and database management.

The separation of these functions provides:

- Better organization.
- Easy maintenance.
- Easy testing.
- Improved security.
- Reusability of components.
- Independent modification of the ML model.
- Easier integration with external systems.

The predictive analytics component can be updated or retrained without changing the complete application.

3. Draw the Software Architecture Diagram



4. Explain Components and Their Interactions

4.1 Describe the purpose and responsibilities of each architectural component.

The presentation layer provides the interface through which users interact with the system.
Functions:

- Login page.
- Dashboard.

- Land data entry.
- Acquisition status display.
- Prediction results.
- Risk visualization.
- Reports and alerts.

4.2 Explain how the components communicate and exchange data.

This layer contains the main business rules of the system.
Functions:

- Manage land records.
- Validate data.
- Monitor acquisition progress.
- Manage compensation information.
- Manage approval status.
- Manage legal information.
- Control user permissions.
- Generate reports.

This is the main analytical component of the system.
Functions:

- Process historical data.
- Clean and prepare data.
- Analyze delay-related factors.
- Apply machine learning algorithms.
- Calculate delay probability.
- Classify cases according to risk.

Example
Acquisition Data
↓
Data Preprocessing
↓
ML Prediction Model
↓
Delay Probability
↓
Risk Classification
↓
Low / Medium / High

Data Access Layer
The data access layer connects the application with the database.
Functions:

- Insert data.
- Retrieve data.
- Update data.
- Delete authorized data.
- Execute database queries.
- Maintain data consistency.

Database Layer
The database stores all important system information.
It contains:

- Land details.
- Landowner details.
- Acquisition records.
- Compensation details.
- Approval information.
- Legal dispute information.
- Historical data.
- Prediction results.
- User information.
- Audit logs.

Notification Service
The notification service sends alerts when a high-risk acquisition case is identified.
High-Risk Case
↓
Notification Service
↓
Email / SMS / Dashboard Alert
↓
Responsible Officer

External Systems
The system can communicate with:

- Government land databases.
- GIS systems.
- Government approval systems.
- Email services.
- SMS gateways.

Communication can be performed through secure APIs.

4.8 Illustrate the overall workflow of the system.

User Login
↓
Enter / Update Land Data
↓
Validate Data
↓
Store Data
↓
Predictive Analysis
↓
Calculate Delay Probability
↓
Classify Risk
↓



5. Discuss Quality Attributes

5.1 Scalability

The system should support an increasing number of users, projects, and land acquisition records.
Additional application servers can be added when the number of users increases.
Benefit: The system can grow without requiring a complete redesign.

5.2 Security

Security can be provided through:

- User authentication.
- Role-based access control.
- Password hashing.
- HTTPS communication.
- Database access control.
- Audit logging.
- Data privacy mechanisms.

Only authorized users should be able to access sensitive land and legal information.

5.3 Performance

The system should provide fast responses for normal operations.
Performance can be improved using:

- Database indexing.
- Optimized queries.
- Caching.
- Load balancing.
- Efficient ML models.
- Asynchronous notifications.

5.4 Reliability

Reliability can be achieved through:

- Data validation.
- Error handling.
- Database backups.
- Transaction management.
- Prediction model validation.

- Recovery mechanisms.

This ensures that important acquisition information is not easily lost or corrupted.

5.5 Maintainability

The layered architecture makes the system easier to maintain.
For example, the ML model can be replaced or upgraded without modifying the entire user interface or database.
Each layer can be tested and maintained independently.

5.6 Availability

The system should remain available to authorized users whenever required.
Availability can be improved using:

- Backup servers.
- Database backup.
- Recovery mechanisms.
- System monitoring.
- Fault detection.
- Redundant services.

6. Justify Architectural Decisions

6.1 Explain the rationale behind your architectural choices.

Layered architecture is selected because it provides a clear separation between:

- User interface.
- Business logic.
- Predictive analytics.
- Data access.
- Database.

This makes the system easier to develop, test, maintain, and extend.

6.2 Discuss the trade-offs considered while selecting the architecture.

The predictive model may need to be retrained when new acquisition data becomes available.
New Historical Data
↓
Data Preprocessing
↓
Model Training
↓
Model Evaluation
↓
Updated ML Model
↓
Improved Delay Prediction
Therefore, keeping the ML component separate makes future model improvements easier.

6.3 Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Architectural Decision	Advantages	Disadvantages
Layered Architecture	Easy maintenance and testing	Additional communication between layers
Separate ML Component	Easy model updating	Requires additional service management
Central Database	Simple data management	Can become a performance bottleneck
REST API	Easy integration	Network communication overhead
Role-Based Security	Better data protection	Requires permission management

Future Enhancements

The architecture can be extended to include:

- GIS-based land mapping.
- Mobile application.
- Real-time acquisition monitoring.
- Advanced machine learning models.
- Integration with more government databases.
- Automated document verification.
- Cloud deployment.
- AI-based recommendations for preventing delays.
- Advanced analytics and visualization.

Conclusion

The Layered Architecture with Client-Server and a separate Predictive Analytics component is suitable for the Predictive Analytics System for Early Detection of Land Acquisition Delays. It provides separation of responsibilities, secure data management, efficient prediction processing, easy maintenance, and scalability. The architecture can also support future integration with GIS, government databases, mobile applications, and advanced machine learning models.